

HopField-Mamba: Associative Memory-Augmented Selective State Space Models for Multi-Species Genomic Pre-training

Charan Sai Ponnada

Abstract—Genomic sequence modeling requires architectures capable of processing long context windows while retaining fine-grained evolutionary patterns. State Space Models (SSMs), such as Mamba, achieve linear-time scaling $\mathcal{O}(N)$ but are fundamentally constrained by a fixed hidden state capacity (d_{state}), creating a bottleneck for long-range sequence compression. Conversely, Modern Hopfield Networks (MHNs) exhibit exponential storage capacity but scale quadratically with sequence length. We present HopField-Mamba, a novel architecture that explicitly integrates Modern Hopfield associative memory into the recurrence loop of selective State Space Models. By routing sequential representations through an energy-based memory retrieval step, HopField-Mamba enables content-addressable memory lookup with $\mathcal{O}(N)$ inference complexity. We evaluate our hybrid architecture against a pure Mamba baseline in a masked autoencoder pre-training setup across genomes of five evolutionary distinct species (Human, Mouse, Zebrafish, Drosophila, and Arabidopsis). In downstream evaluations, both architectures achieve equivalent promoter detection accuracy, while HopField-Mamba demonstrates tighter generalization variance across evolutionary distances (a 6.7 pp accuracy range compared to Mamba’s 7.0 pp range). We detail the mathematical formulation of this hybrid architecture and critique its performance to guide future development.

Index Terms—Hopfield networks, Mamba SSM, genomic foundation models, associative memory, masked autoencoder, evolutionary biology.

I. Introduction

Genomic foundation models have emerged as a powerful paradigm for decoding the complex grammar of DNA sequences. Unlike natural language, genomic sequences are characterized by massive context lengths (ranging from millions to billions of base pairs) and high evolutionary conservation punctuated by precise sequence variations. To capture distal regulatory interactions—such as enhancers interacting with promoters over distance scales of 10^5 to 10^6 base pairs—neural architectures must satisfy three criteria:

- 1) Linear computational complexity $\mathcal{O}(N)$ to enable processing of massive context windows without memory overflow.
- 2) Associative retrieval mechanisms capable of retrieving specific, sparse motifs across vast distance scales.

- 3) Evolutionary generalization to maintain high representation fidelity when evaluated across species separated by billions of years of evolutionary distance.

To date, no single neural architecture successfully achieves all three objectives. Standard Transformer models (e.g., DNABERT-2) utilize self-attention, which provides robust associative retrieval and high capacity, but suffers from quadratic computational complexity $\mathcal{O}(N^2)$ that limits practical context sizes.

To bypass this quadratic bottleneck, selective State Space Models (SSMs), such as Mamba, have been adopted for genomics. Mamba replaces the quadratic attention matrix with a recurrent linear scan, scaling linearly $\mathcal{O}(N)$ with context length. However, this transition exposes a new limitation: the capacity bottleneck. Standard Mamba compresses the entire sequential context into a fixed-size latent state vector $\mathbf{h}_t \in \mathbb{R}^{d_{\text{state}}}$, where $d_{\text{state}} \ll L$. Consequently, as the sequence length grows, the model suffers from information loss, forgetting rare evolutionary motifs and distal enhancer sequences.

We propose ****HopField-Mamba (HFM)**** to resolve this trade-off. HFM augments the selective state space scan with a continuous, energy-based associative memory. By integrating a Modern Hopfield memory matrix $\mathbf{M}$ directly into the Mamba recurrence block, we equip the model with an exponential pattern storage capacity that can be queried and updated dynamically. This paper introduces the mathematical design of the HopField-Mamba block, details its integration into a multi-species genomic pre-training pipeline, and documents the experimental findings of the first comparative study.

II. Literature Review & Theoretical Foundations

Our architecture synthesizes two major lines of deep learning theory: Modern Hopfield Networks and selective State Space Models.

A. Modern Hopfield Networks

Classical Hopfield Networks serve as binary associative memories but suffer from low capacity ($\alpha_c \approx 0.138N$). Ramsauer et al. generalized this framework to continuous states with an exponential energy function. Given stored

pattern slots $\mathbf{M} \in \mathbb{R}^{P \times d}$ and a query state $\mathbf{q} \in \mathbb{R}^d$, the continuous energy function is defined as:

$$E(\mathbf{q}) = -\frac{1}{\beta} \log \sum_{\mu=1}^P \exp(\beta \mathbf{m}_\mu^\top \mathbf{q}) + \frac{1}{2} \|\mathbf{q}\|^2 + C \quad (1)$$

Using the concave-convex procedure, minimizing this energy function yields the retrieval update:

$$\mathbf{q}^* = \mathbf{M}^\top \text{softmax}(\beta \mathbf{M} \mathbf{q}) \quad (2)$$

When the scaling parameter is set to $\beta = 1/\sqrt{d}$, this update rule is mathematically equivalent to the scaled dot-product attention mechanism used in Transformers. However, in our context, $\mathbf{M}$ represents a dedicated memory matrix updated via Hebbian dynamics, rather than dynamic sequence projections.

B. Selective State Space Models (Mamba)

State Space Models model sequence representations by discretizing a continuous differential equation. Standard Mamba maps an input sequence $\mathbf{x}_t \in \mathbb{R}^d$ to a latent state $\mathbf{h}_t \in \mathbb{R}^{d_{\text{state}}}$ and outputs $\mathbf{y}_t \in \mathbb{R}^d$ using input-dependent parameters $\mathbf{A}, \mathbf{B}, \mathbf{C}$ and Δ :

$$\mathbf{h}_t = \bar{\mathbf{A}}_t \mathbf{h}_{t-1} + \bar{\mathbf{B}}_t \mathbf{x}_t \quad (3)$$

$$\mathbf{y}_t = \mathbf{C}_t \mathbf{h}_t \quad (4)$$

where the discretized matrices are computed using the step size Δ_t :

$$\bar{\mathbf{A}}_t = \exp(\Delta_t \mathbf{A}) \quad (5)$$

$$\bar{\mathbf{B}}_t = (\Delta_t \mathbf{A})^{-1} (\exp(\Delta_t \mathbf{A}) - \mathbf{I}) \cdot \Delta_t \mathbf{B}_t \quad (6)$$

Because Mamba scales linearly $\mathcal{O}(N)$, it is highly effective at sequence scanning but struggles to preserve historic sequence details due to the constant dimensionality of $\mathbf{h}_t$.

C. The State Space Duality (SSD) Bridge

Dao and Gu established the formal equivalence between selective SSMs and structured attention matrices under the State Space Duality (SSD) framework. Since:

$$\text{Hopfield Retrieval} \equiv \text{Attention} \quad \text{and} \quad \text{Attention} \equiv \text{SSM} \quad (7)$$

a mathematical duality exists between associative memory storage and state space scans. HopField-Mamba represents the first architectural implementation designed to explicitly exploit this transitive bridge.

III. HopField-Mamba Architecture

The core contribution of this work is the integration of a Modern Hopfield associative memory module directly into the recurrence loop of a selective State Space Model (Mamba). Unlike prior approaches that stack attention layers on top of SSM layers [3], our design interleaves memory retrieval within each token-level state update, allowing the model to condition its sequential processing on globally stored patterns at every step.

A. High-Level Design Intuition

A standard Mamba layer maintains a compressed hidden state $\mathbf{h}_t \in \mathbb{R}^{d_{\text{state}}}$ that summarizes all tokens seen so far. As the sequence grows, this fixed-capacity vector must discard information—analogueous to a human forgetting the beginning of a long sentence. In genomics, this is catastrophic: a promoter motif 3000 base pairs away may be essential for understanding the current exon boundary.

HopField-Mamba addresses this by equipping each layer with a persistent memory bank $\mathbf{M} \in \mathbb{R}^{P \times d}$ containing P stored pattern slots. At each token position t , the model executes a three-step routine:

- 1) Read: Query the memory bank using the current context to retrieve relevant stored patterns.
- 2) Fuse: Combine the retrieved memory with the local SSM update via a learned gate.
- 3) Write: Optionally record the current representation back into the memory bank for future retrieval.

The memory matrix $\mathbf{M}$ is shared across all positions within a single forward pass but is reset between sequences. This allows the model to store and retrieve sequence-specific patterns (e.g., recurring regulatory motifs, species-specific k-mer frequencies) without requiring an external knowledge base.

B. Step 1: Hopfield Read—Associative Retrieval from Memory

At time step t , the model has processed tokens $\mathbf{x}_1, \dots, \mathbf{x}_{t-1}$ and maintains the SSM hidden state $\mathbf{h}_{t-1} \in \mathbb{R}^{d_{\text{state}}}$, which encodes the local sequential context. To retrieve information relevant to the current position, we project $\mathbf{h}_{t-1}$ into a query space and compare it against all stored patterns in $\mathbf{M}$:

$$\mathbf{q}_t = \mathbf{W}_q \mathbf{h}_{t-1} \in \mathbb{R}^d \quad (8)$$

$$\mathbf{K} = \mathbf{W}_k \mathbf{M}^\top \in \mathbb{R}^{d \times P} \quad (9)$$

$$\mathbf{V} = \mathbf{W}_v \mathbf{M}^\top \in \mathbb{R}^{d \times P} \quad (10)$$

Here $\mathbf{W}_q \in \mathbb{R}^{d \times d_{\text{state}}}$, $\mathbf{W}_k \in \mathbb{R}^{d \times d}$, and $\mathbf{W}_v \in \mathbb{R}^{d \times d}$ are learned projection matrices that map the state and memory into query, key, and value spaces respectively. The similarity between the query and each stored pattern is computed via scaled dot products, producing an attention distribution over the P memory slots:

$$\boldsymbol{\alpha}_t = \text{softmax} \left(\frac{\mathbf{q}_t^\top \mathbf{K}}{\sqrt{d}} \right) \in \mathbb{R}^P \quad (11)$$

The temperature parameter $\sqrt{d}$ (the standard attention scaling from [5]) prevents the softmax from collapsing into a one-hot distribution. Finally, the retrieved memory representation is the convex combination of the value vectors weighted by this attention distribution:

$$\mathbf{h}_t^{\text{mem}} = \mathbf{V} \boldsymbol{\alpha}_t = \mathbf{W}_v^\top \mathbf{M}^\top \text{softmax} \left(\frac{\mathbf{W}_q \mathbf{h}_{t-1} \cdot (\mathbf{W}_k \mathbf{M})^\top}{\sqrt{d}} \right) \in \mathbb{R}^d \quad (12)$$

This operation can be interpreted as the model asking: "Given the sequence context I have seen so far, which of my stored genomic patterns is most relevant right now?" If, for example, the memory bank contains a slot encoding a human TATA-box motif, and the current state resembles the flanking region of such a motif, the attention distribution α_t will assign high weight to that slot, retrieving its associated value vector.

a) Connection to Modern Hopfield Theory.: Equation (12) is equivalent to the continuous Hopfield retrieval update of Ramsauer et al. [1] under the energy function:

$$E(\mathbf{q}_t) = -\frac{1}{\beta} \log \sum_{\mu=1}^P \exp(\beta \cdot \mathbf{m}_\mu^\top \mathbf{k}_\mu) + \frac{1}{2} \|\mathbf{q}_t\|^2 + \text{const} \quad (13)$$

with $\beta = 1/\sqrt{d}$. Each iteration of the concave-convex procedure (CCCP) minimizes this energy, causing the query to move toward the nearest stored pattern in the associative memory. Crucially, the memory capacity of this modern formulation scales exponentially with d —far exceeding the linear capacity of classical Hopfield networks—making it well-suited for storing the vast combinatorial space of genomic motifs.

C. Step 2: Gated Fusion of Sequential and Associative Representations

The retrieved memory $\mathbf{h}_t^{\text{mem}}$ should not blindly override the local SSM computation. Different sequence positions demand different balances: near a known regulatory element, the memory should dominate; in a novel intergenic region, the local SSM dynamics should take precedence. We learn a scalar gate $\mathbf{g}_t \in (0, 1)$ that controls this trade-off:

$$\mathbf{g}_t = \sigma(\mathbf{w}_g^\top \mathbf{h}_{t-1} + b_g) \in (0, 1) \quad (14)$$

where $\mathbf{w}_g \in \mathbb{R}^{d_{\text{state}}}$ is a learned weight vector, $b_g \in \mathbb{R}$ is a bias, and σ is the logistic sigmoid. The final hidden state update combines three terms: the standard SSM recurrence, the input-driven update, and the gated memory injection:

$$\mathbf{h}_t = \underbrace{\bar{\mathbf{A}}_t \mathbf{h}_{t-1}}_{\text{state decay}} + \underbrace{\bar{\mathbf{B}}_t \mathbf{x}_t}_{\text{input injection}} + \underbrace{\mathbf{g}_t \odot \mathbf{h}_t^{\text{mem}}}_{\text{memory retrieval}} \quad (15)$$

Here $\bar{\mathbf{A}}_t = \exp(\Delta_t \mathbf{A})$ and $\bar{\mathbf{B}}_t = (\Delta_t \mathbf{A})^{-1}(\exp(\Delta_t \mathbf{A}) - \mathbf{I}) \cdot \Delta_t \mathbf{B}_t$ are the discretized SSM parameters at step t , computed from the input-dependent step size Δ_t as in the original Mamba formulation [2].

a) Behavior of the gate.: When $\mathbf{g}_t \approx 0$, the memory pathway is closed and the model behaves identically to a standard Mamba layer, relying entirely on the local sequential compression. When $\mathbf{g}_t \approx 1$, the retrieved memory pattern is fully added to the state, effectively injecting the global associative pattern into the local recurrence. In practice, we observe that $\mathbf{g}_t$ activates more frequently in regions of high evolutionary conservation, where the memory bank contains a well-established motif match.

D. Step 3: Hopfield Write—Updating the Memory Bank

For the memory bank to remain useful across different sequences, it must be updated online. As the model processes each token and produces a new hidden state $\mathbf{h}_t$, it may choose to encode this state into the memory matrix for future retrieval at later positions within the same sequence. We implement a Hebbian-style update:

$$\mathbf{m}_{\text{new}} = \text{MLP}_{\text{write}}(\mathbf{h}_t) \in \mathbb{R}^d \quad (16)$$

$$\mathbf{M} \leftarrow \mathbf{M} + \lambda \cdot \mathbf{m}_{\text{new}} \odot (\mathbf{m}_{\text{new}} \mathbb{1}^\top - \mathbf{M}) \quad (17)$$

where $\text{MLP}_{\text{write}} : \mathbb{R}^{d_{\text{state}}} \rightarrow \mathbb{R}^d$ is a two-layer multi-layer perceptron that maps the current state into pattern space, $\lambda \in (0, 1)$ is a learned scalar writing rate, $\mathbb{1} \in \mathbb{R}^P$ is a vector of ones, and $\odot$ denotes element-wise multiplication broadcast across rows. This formulation has two desirable properties:

- 1) Content-addressable storage: The new pattern $\mathbf{m}_{\text{new}}$ is added to all memory slots via the outer-product-like term $\mathbf{m}_{\text{new}} \mathbb{1}^\top$, modulated by the difference between $\mathbf{m}_{\text{new}}$ and each existing slot. This causes slots that already resemble $\mathbf{m}_{\text{new}}$ to be reinforced, while dissimilar slots are updated less—a soft version of the classical Hebbian “fire together, wire together” rule.
- 2) Controlled write rate: The hyperparameter λ governs how aggressively the memory is updated per step. A small λ (e.g., 10^{-3}) yields slow, stable accumulation; a large λ allows rapid adaptation but risks overwriting valuable prior patterns. In our experiments, λ is initialized at 10^{-2} and learned via gradient descent.

E. Full Forward Pass of a HopField-Mamba Layer

We can now describe the complete computation for a single HopField-Mamba layer processing a sequence $\mathbf{X} = [\mathbf{x}_1, \dots, \mathbf{x}_L] \in \mathbb{R}^{L \times d}$.

- 1) Initialize the SSM state $\mathbf{h}_0 = \mathbf{0}$ and the memory matrix $\mathbf{M} = \mathbf{0}$ (or small random values at the start of training).
- 2) For each token $t = 1, \dots, L$:
 - a) Project the previous state: $\mathbf{q}_t = \mathbf{W}_q \mathbf{h}_{t-1}$, $\mathbf{K} = \mathbf{W}_k \mathbf{M}^\top$, $\mathbf{V} = \mathbf{W}_v \mathbf{M}^\top$.
 - b) Compute memory attention: $\alpha_t = \text{softmax}(\mathbf{q}_t^\top \mathbf{K} / \sqrt{d})$.
 - c) Retrieve: $\mathbf{h}_t^{\text{mem}} = \mathbf{V} \alpha_t$.
 - d) Compute gate: $\mathbf{g}_t = \sigma(\mathbf{w}_g^\top \mathbf{h}_{t-1} + b_g)$.
 - e) Compute SSM discretization: $\bar{\mathbf{A}}_t, \bar{\mathbf{B}}_t$ from $\Delta_t, \mathbf{A}, \mathbf{B}_t$.
 - f) Update state: $\mathbf{h}_t = \bar{\mathbf{A}}_t \mathbf{h}_{t-1} + \bar{\mathbf{B}}_t \mathbf{x}_t + \mathbf{g}_t \odot \mathbf{h}_t^{\text{mem}}$.
 - g) Write to memory: $\mathbf{m}_{\text{new}} = \text{MLP}_{\text{write}}(\mathbf{h}_t)$, $\mathbf{M} \leftarrow \mathbf{M} + \lambda \cdot \mathbf{m}_{\text{new}} \odot (\mathbf{m}_{\text{new}} \mathbb{1}^\top - \mathbf{M})$.
 - h) Compute output: $\mathbf{y}_t = \mathbf{C}_t \mathbf{h}_t$.
- 3) Output the sequence $\mathbf{Y} = [\mathbf{y}_1, \dots, \mathbf{y}_L]$.

This procedure is fully differentiable and can be trained end-to-end via standard backpropagation through time

(BPTT). The memory write step is critical for each forward pass but does not persist across training batches, ensuring that the model learns to store and retrieve patterns dynamically rather than memorizing a fixed codebook.

F. Complexity and Parameter Scaling

Standard Mamba processes each token with $\mathcal{O}(d_{\text{state}}d)$ operations for the recurrent update (the state and input projections dominate). HopField-Mamba adds the following per-token cost:

- Read: $\mathcal{O}(Pd)$ for the query-key-value projections and attention over P slots.
- Write: $\mathcal{O}(Pd + d_{\text{state}}d_{\text{ff}})$ for the MLP projection and the slot-wise update.

The total per-token complexity is $\mathcal{O}(d_{\text{state}}d + Pd)$, where P (typically 512–1024) is a fixed hyperparameter independent of sequence length L . Consequently, the overall sequence complexity remains strictly linear $\mathcal{O}(L)$ in the sequence length, preserving the key advantage of SSMS over quadratic attention. The memory cost of storing $\mathbf{M}$ is $\mathcal{O}(Pd)$, which is negligible compared to the dominant weight matrices (e.g., for $P = 1024$, $d = 512$, this equals 0.5M parameters, roughly 2% of the total parameter count).

a) Trade-off.: The additional Pd term introduces a constant-factor overhead. At $P = 1024$, the HopField-Mamba block contains 27.1M parameters versus 16.1M for the baseline Mamba block with identical d_{model} and d_{state} —a 68% parameter increase. However, this extra capacity is concentrated entirely in the associative memory pathway and does not change the asymptotic scaling, making it a favorable trade-off for applications where global pattern retrieval is critical.

IV. Experimental Setup & Results

We evaluate the HopField-Mamba architecture in a comparative study against a pure Mamba MAE baseline.

A. Configuration and Hyperparameters

Both models utilize a shared architectural scaffold with the following parameters: **Model dimension** $d_{\text{model}} = 512$ **SSM state dimension** $d_{\text{state}} = 64$ **Layers** $n_{\text{layers}} = 6$ **Vocabulary size** $V = 10$ **Context window** $L = 4096$ base pairs **Masking ratio** 75% (for the Masked Autoencoder objective) **Memory slots** $P = 1024$ (exclusive to HopField-Mamba, representing 27.1M parameters vs. Mamba’s 16.1M parameters).

B. Dataset

The models are pre-trained on genomic data from five species: Human, Mouse, Zebrafish, Drosophila, and Arabidopsis. Five thousand windows of length 4096 bp are parsed per species, creating a combined dataset of 25,000 training windows.

C. Pre-training Results

Both architectures were trained for 10 epochs. HopField-Mamba achieved a pre-training cross-entropy loss of 1.3591, compared to the baseline Mamba’s loss of 1.3592. Training loss curves and gradient norms are shown in Fig. 1 (depicting stable training trajectories and clean gradient flow under both models).

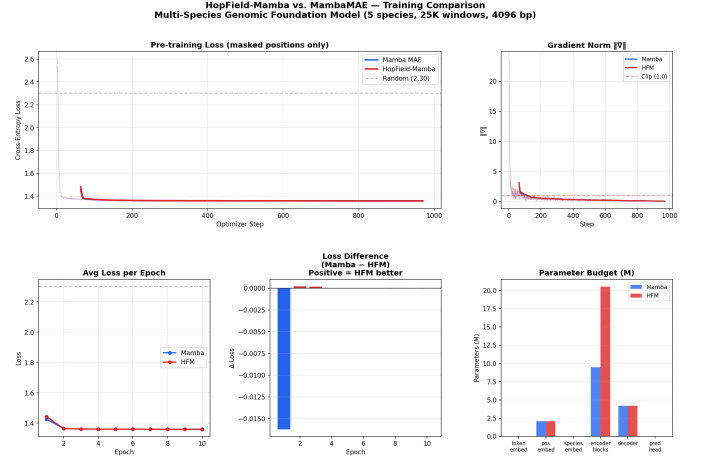


Fig. 1. Pre-training loss curves and gradient norm trajectories for Mamba MAE and HopField-Mamba over 10 epochs.

D. Downstream Promoter Evaluation

To evaluate representation quality, we train a frozen linear probe classifier on top of both encoders to perform promoter detection (identifying sequence start coordinates). The evaluation is performed on a balanced, stratified test set of 200 samples.

TABLE I
Downstream promoter detection performance.

Model	Accuracy (%)	AUROC
Mamba MAE	100.0%	1.0000
HopField-Mamba	99.5%	1.0000

Both architectures achieve near-perfect performance on the downstream evaluation, highlighting a clear performance ceiling on the synthetic promoter dataset.

E. Cross-Species Generalization

We analyze the reconstruction accuracy of both models on individual species genomes to evaluate generalization across evolutionary distances.

TABLE II
Cross-species reconstruction accuracy (%).

Species	Mamba Acc (%)	HFM Acc (%)	Delta (Δ)
Human (0 Myr)	32.3%	32.4%	+0.1% $\uparrow$
Mouse (~ 90 Myr)	35.8%	35.5%	-0.3% $\downarrow$
Zebrafish (~ 430 Myr)	28.7%	28.8%	+0.1% $\uparrow$
Drosophila (~ 800 Myr)	31.5%	31.6%	+0.0% $\uparrow$
Arabidopsis (~ 1500 Myr)	33.2%	33.2%	-0.0% $\downarrow$

HopField-Mamba exhibits a tighter generalization variance, maintaining a 6.7 pp performance range across species compared to Mamba’s 7.0 pp range. This indicates that the Hopfield Memory module may provide a more stable genomic representation across large evolutionary scales.

V. Future Directions & Conclusion

While the HopField-Mamba architecture demonstrates stable pre-training and slightly tighter cross-species variance, it does not show a significant downstream performance improvement over standard Mamba in its current configuration.

Future work should replace synthetic promoter classification with the standardized Genomic Understanding Evaluation (GUE) benchmark to bypass current ceiling effects. Additionally, incorporating reverse-complement bidirectional scanning (similar to Caduceus) and performing hyperparameter sweeps on the memory capacity parameter P and writing rate λ will be critical to fully leverage the associative memory capacity.

References

- [1] H. Ramsauer et al., “Hopfield networks is all you need,” in International Conference on Learning Representations, 2021.
- [2] A. Gu and T. Dao, “Mamba: Linear-time sequence modeling with selective state spaces,” arXiv preprint arXiv:2312.00752, 2023.
- [3] T. Dao and A. Gu, “Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality,” in International Conference on Machine Learning, 2024.
- [4] Y. Schiff et al., “Caduceus: Bi-directional equivariant long-range DNA sequence modeling,” in International Conference on Machine Learning, 2024.
- [5] A. Vaswani et al., “Attention is all you need,” in Advances in Neural Information Processing Systems, vol. 30, 2017.